Translation Validation of Loop Invariant Compiler Optimizations

Chandan Karfa



ভাৰতীয় প্ৰযুক্তিবিদ্যা প্ৰতিষ্ঠান গুৱাহাটী
भारतीय प्रौद्योगिकी संस्थान गुवाहाटी
INDIAN INSTITUTE OF TECHNOLOGY GUWAHATI

Outline

Outline of the Verification Method

Outline of the Verification method:

- Let M_0 and M_1 be the FSMDs corresponding to the behaviours before and after of any phase, respectively.

Steps:

1. Find the path cover, P_0, in M_0.
2. For every path in P_0, find an equivalent path in M_1.
3. Repeat Step 1 and 2 with FSMDs interchanged.

Problem with finding Path Cover:

- **Loops** in FSMD.

Solutions

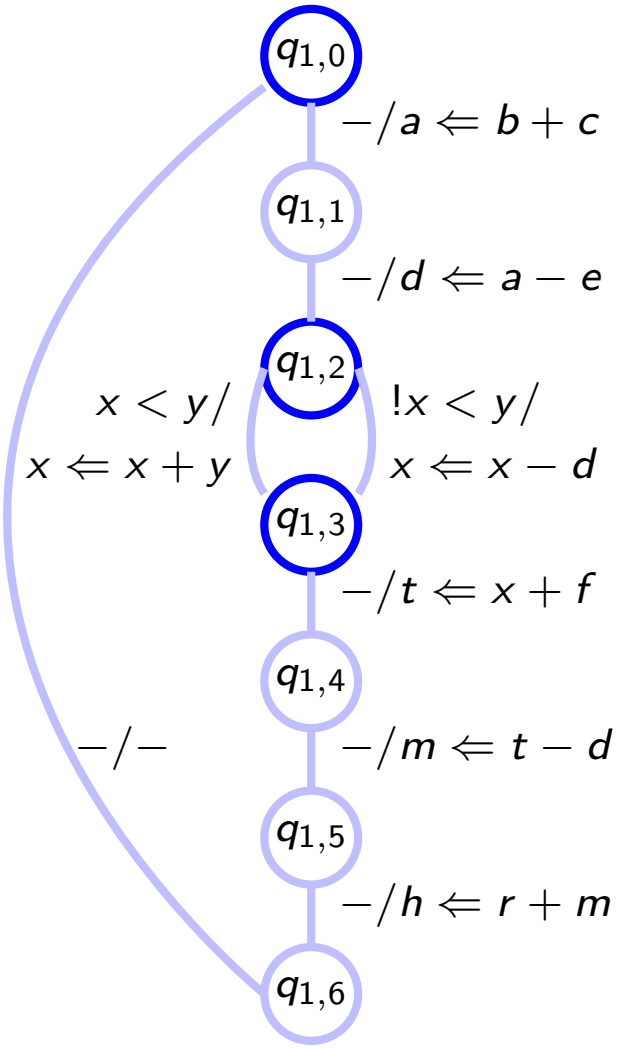
Solutions:

- Based on **Floyd–Hoare's method of program verification**.
- Put cutpoints in FSMD such that each loop is cut at least one cutpoint.
- Set of paths from one cutpoint to another without having any intermediate cutpoint is a path cover.

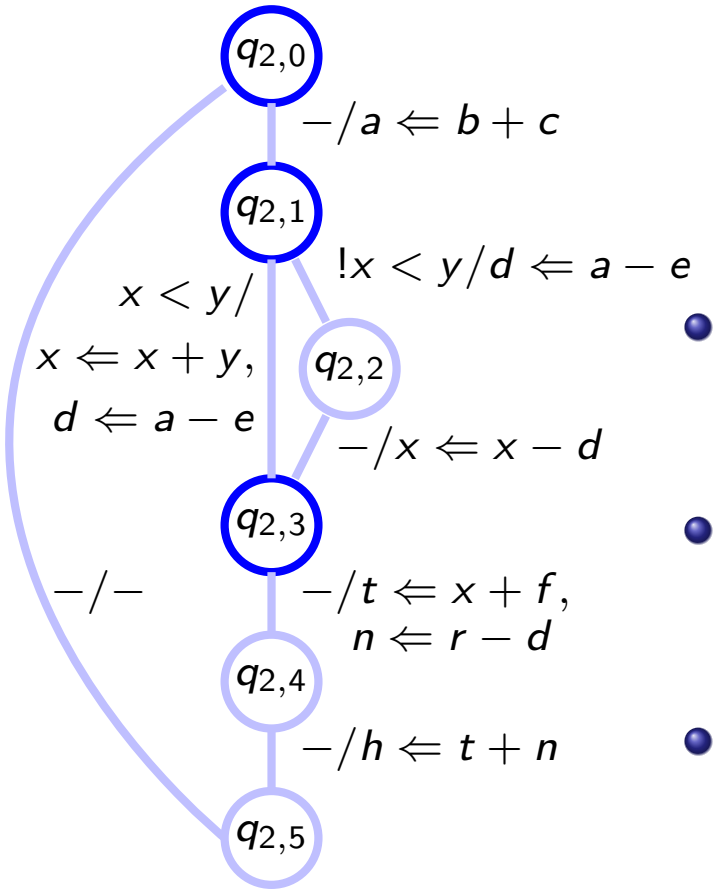
Cutpoints: Defined as follows in our work

1. Reset state.
2. State with more than one outward transition.

Equivalence checking of FSMDs: A basic example



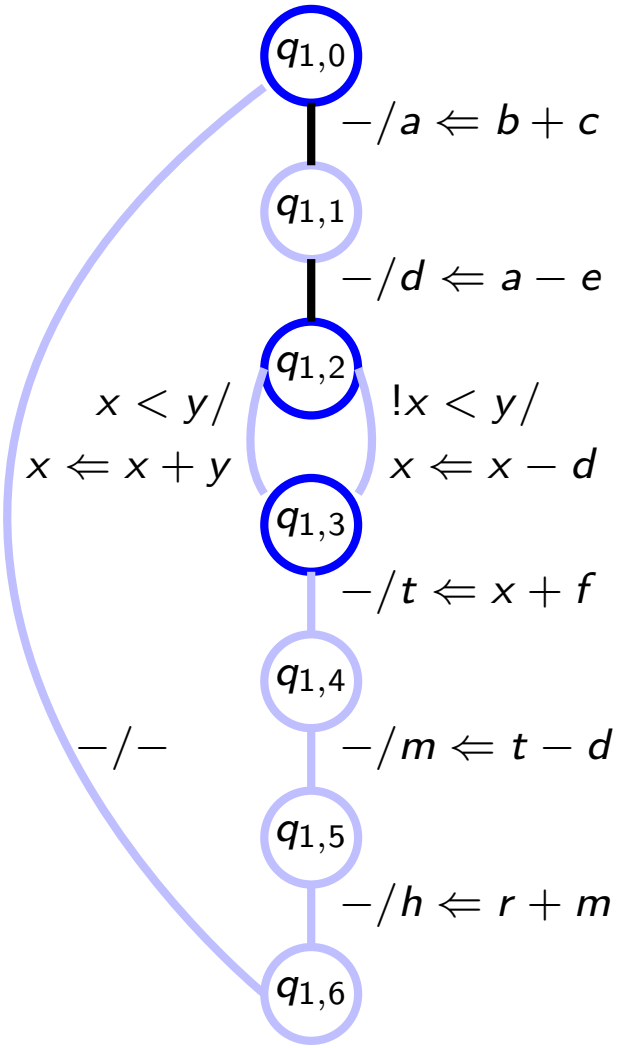
(a) M_1



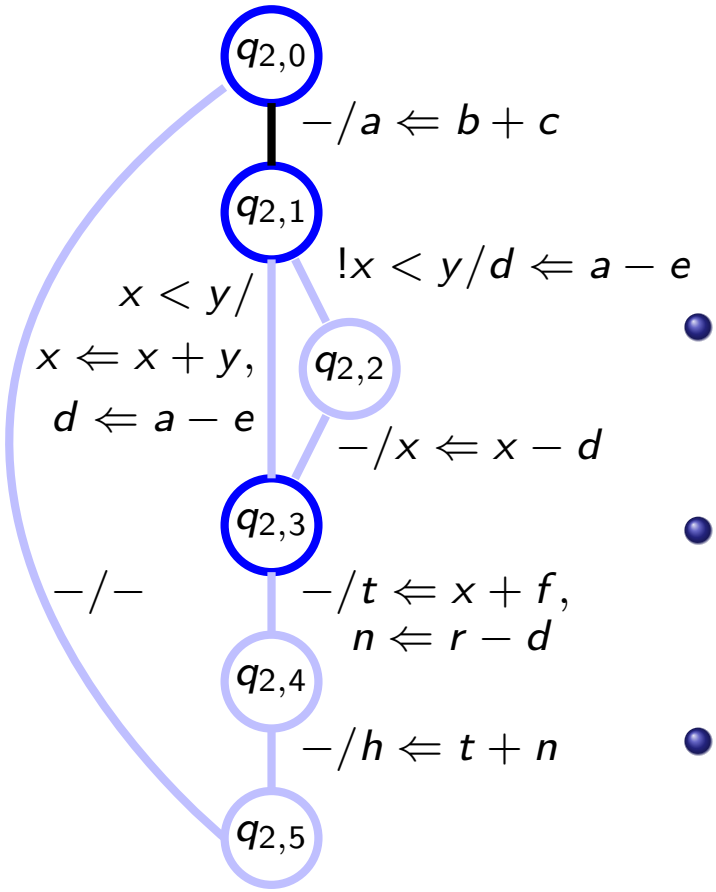
(b) M_2

- Two Finite State Machines with Datapath (FSMDs) M_1 and M_2 are equivalent if for every path in P_1 there is an equivalent path in P_2 and vice versa
- Code transformations can make this job difficult
- Paths may be extended, and the path covers are updated accordingly
- $\{q_{1,0} \overset{x<y}{\Longrightarrow} q_{1,3} \simeq q_{2,0} \overset{x<y}{\Longrightarrow} q_{2,3}, q_{1,0} \overset{!x<y}{\Longrightarrow} q_{1,3} \simeq q_{2,0} \overset{!x<y}{\Longrightarrow} q_{2,3},$ $q_{1,3} \Longrightarrow q_{1,0} \simeq q_{2,3} \Longrightarrow$

Equivalence checking of FSMDs: A basic example



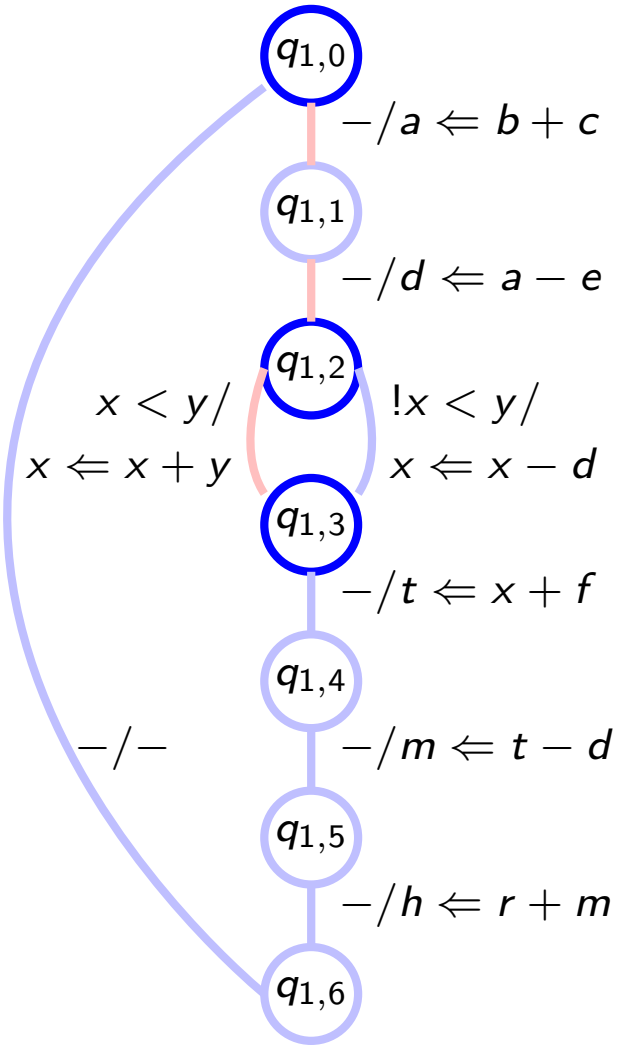
(a) M_1



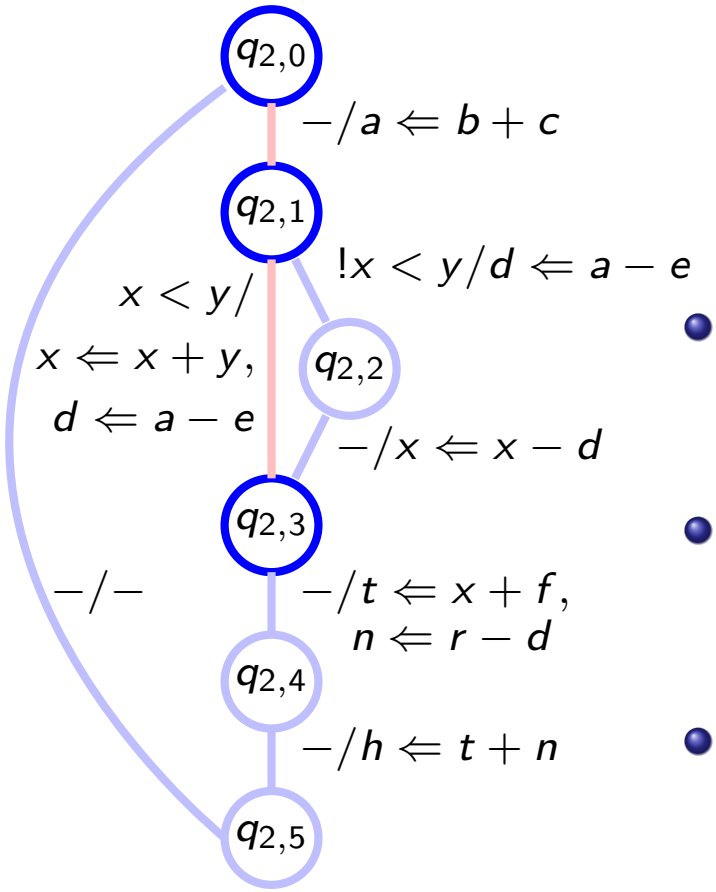
(b) M_2

- Two Finite State Machines with Datapath (FSMDs) M_1 and M_2 are equivalent if for every path in P_1 there is an equivalent path in P_2 and vice versa
- Code transformations can make this job difficult
- Paths may be extended, and the path covers are updated accordingly
- $\{q_{1,0} \overset{x<y}{\Longrightarrow} q_{1,3} \simeq q_{2,0} \overset{x<y}{\Longrightarrow} q_{2,3}, q_{1,0} \overset{!x<y}{\Longrightarrow} q_{1,3} \simeq q_{2,0} \overset{!x<y}{\Longrightarrow} q_{2,3},$ $q_{1,3} \Longrightarrow q_{1,0} \simeq q_{2,3} \Longrightarrow$

Equivalence checking of FSMDs: A basic example



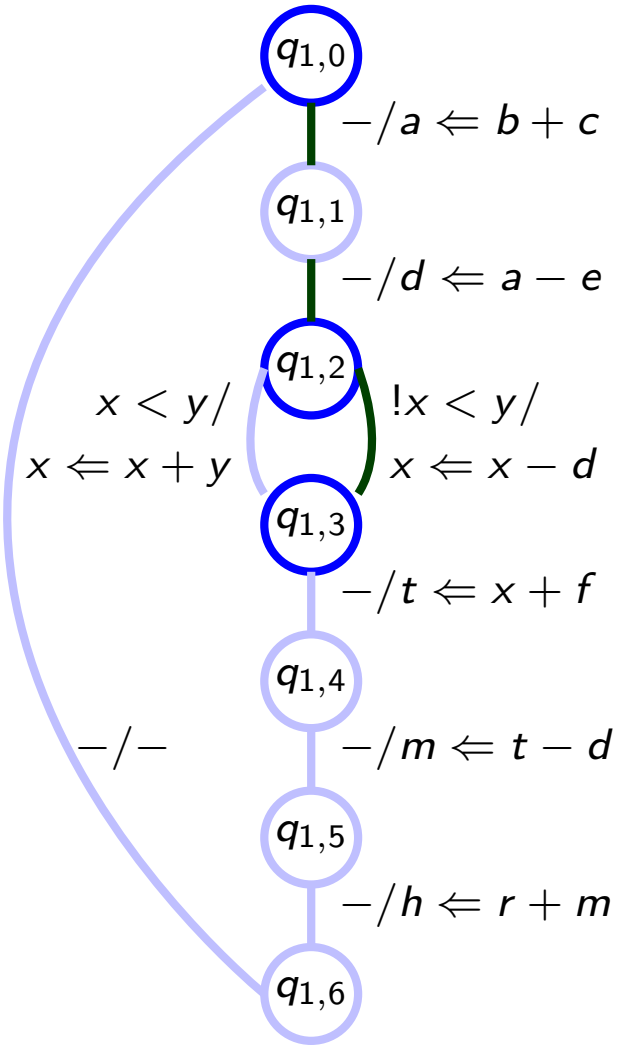
(a) M_1



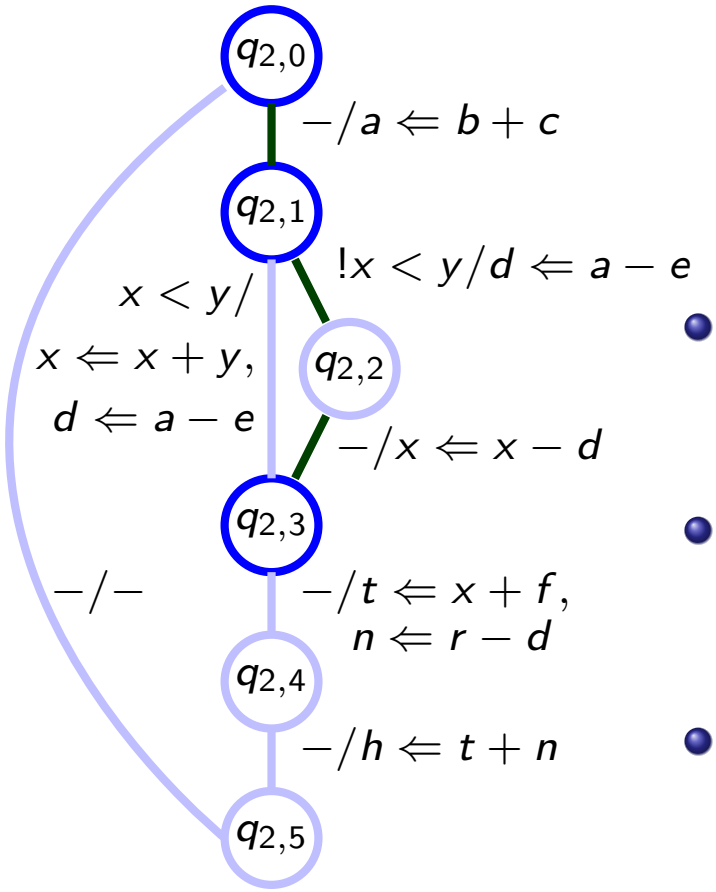
(b) M_2

- Two Finite State Machines with Datapath (FSMDs) M_1 and M_2 are equivalent if for every path in P_1 there is an equivalent path in P_2 and vice versa
- Code transformations can make this job difficult
- Paths may be extended, and the path covers are updated accordingly
- $\{q_{1,0} \overset{x<y}{\Longrightarrow} q_{1,3} \simeq q_{2,0} \overset{x<y}{\Longrightarrow} q_{2,3}, q_{1,0} \overset{!x<y}{\Longrightarrow} q_{1,3} \simeq q_{2,0} \overset{!x<y}{\Longrightarrow} q_{2,3},$ $q_{1,3} \Longrightarrow q_{1,0} \simeq q_{2,3} \Longrightarrow$

Equivalence checking of FSMDs: A basic example



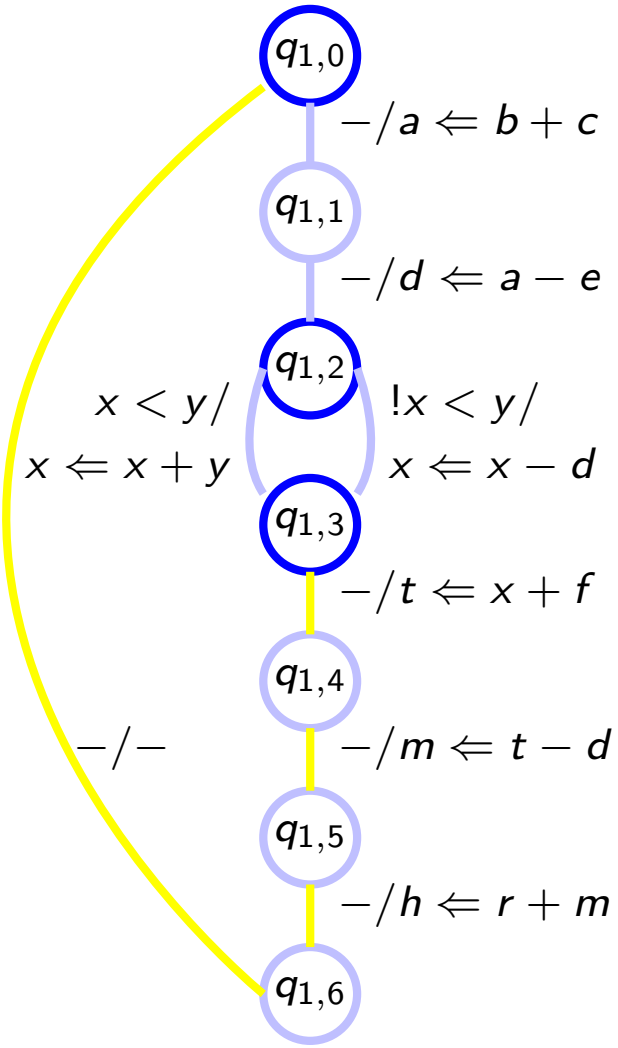
(a) M_1



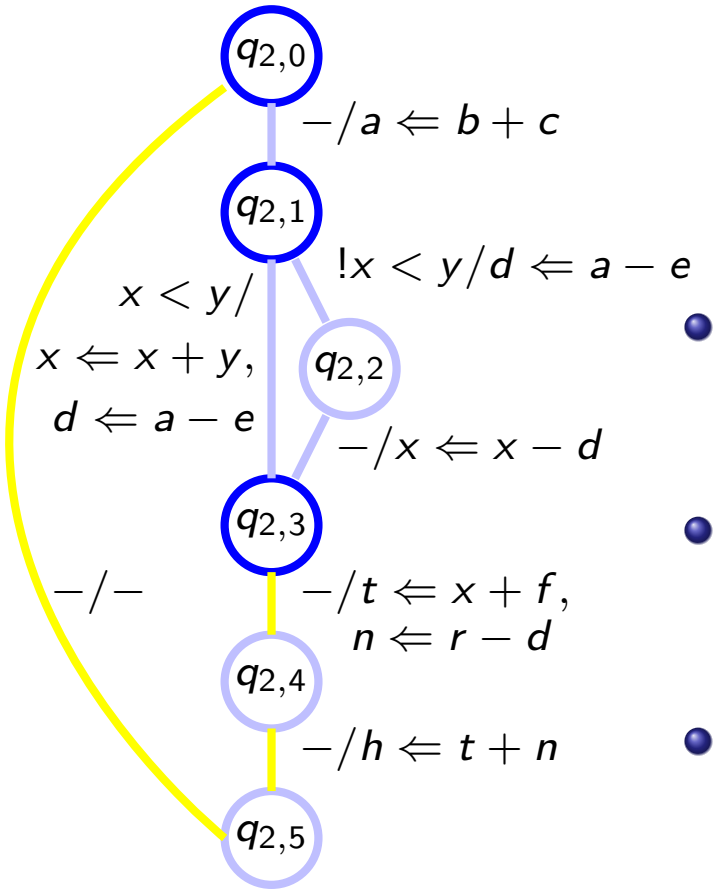
(b) M_2

- Two Finite State Machines with Datapath (FSMDs) M_1 and M_2 are equivalent if for every path in P_1 there is an equivalent path in P_2 and vice versa
- Code transformations can make this job difficult
- Paths may be extended, and the path covers are updated accordingly
- $\{q_{1,0} \overset{x<y}{\Longrightarrow} q_{1,3} \simeq q_{2,0} \overset{x<y}{\Longrightarrow} q_{2,3}, q_{1,0} \overset{!x<y}{\Longrightarrow} q_{1,3} \simeq q_{2,0} \overset{!x<y}{\Longrightarrow} q_{2,3}, q_{1,3} \Longrightarrow q_{1,0} \simeq q_{2,3} \Longrightarrow$

Equivalence checking of FSMDs: A basic example



(a) M_1



(b) M_2

- Two Finite State Machines with Datapath (FSMDs) M_1 and M_2 are equivalent if for every path in P_1 there is an equivalent path in P_2 and vice versa
- Code transformations can make this job difficult
- Paths may be extended, and the path covers are updated accordingly
- $\{q_{1,0} \overset{x<y}{\Longrightarrow} q_{1,3} \simeq q_{2,0} \overset{x<y}{\Longrightarrow} q_{2,3}, q_{1,0} \overset{!x<y}{\Longrightarrow} q_{1,3} \simeq q_{2,0} \overset{!x<y}{\Longrightarrow} q_{2,3}, q_{1,3} \Longrightarrow q_{1,0} \simeq q_{2,3} \Longrightarrow$

Related Work

C. Karfa et al "An equivalence-checking method for scheduling verification in high-level synthesis," TCAD, (2008).

- Proposed a path based equivalence checking method of Finite State Machine with Datapaths (FSMDs).
- Capable of verifying several uniform code motion techniques such as renaming and duplicating.
- Gives false negative results for nonuniform code motion.

Related Work

C. Karfa et al ,"Formal verification of code motion techniques using data-flow-driven equivalence checking" TODAES, (2012).

- Use path based equivalence checking method of FSMDs.
- Strong enough to handle both uniform and nonuniform code motion.
- Fails in the case of code motion across a loop.

Related Work

Some other existing works try to increase the scope of code motion validation.

- Nishihara, Tasuku. Formal Verification of High-Level Design Based on Control/Data Separation. Diss. (2010).
- S. Kundu et al, "Translation validation of high-level synthesis", (2011)
- Chi-Hui Lee et al, "Equivalence checking of scheduling with speculative code transformations in high-level synthesis", (2011)
- S. Bandyopadhyay et al, "Translation validation for PRES+ models of parallel behaviours via an FSMD equivalence checker", (2012)
- Tun Li et al, "Translation validation of scheduling in high level synthesis", (2013)
- Alan Leung et al, "C-to-Verilog translation validation." (2015).
- Benjamin Carrion Schafer "Source code error detection in high-level synthesis functional verification", (2016)
- S.Dutta, "Validation of parallelizing transformations of sequential programs." Concurrency and Computation,(2017).

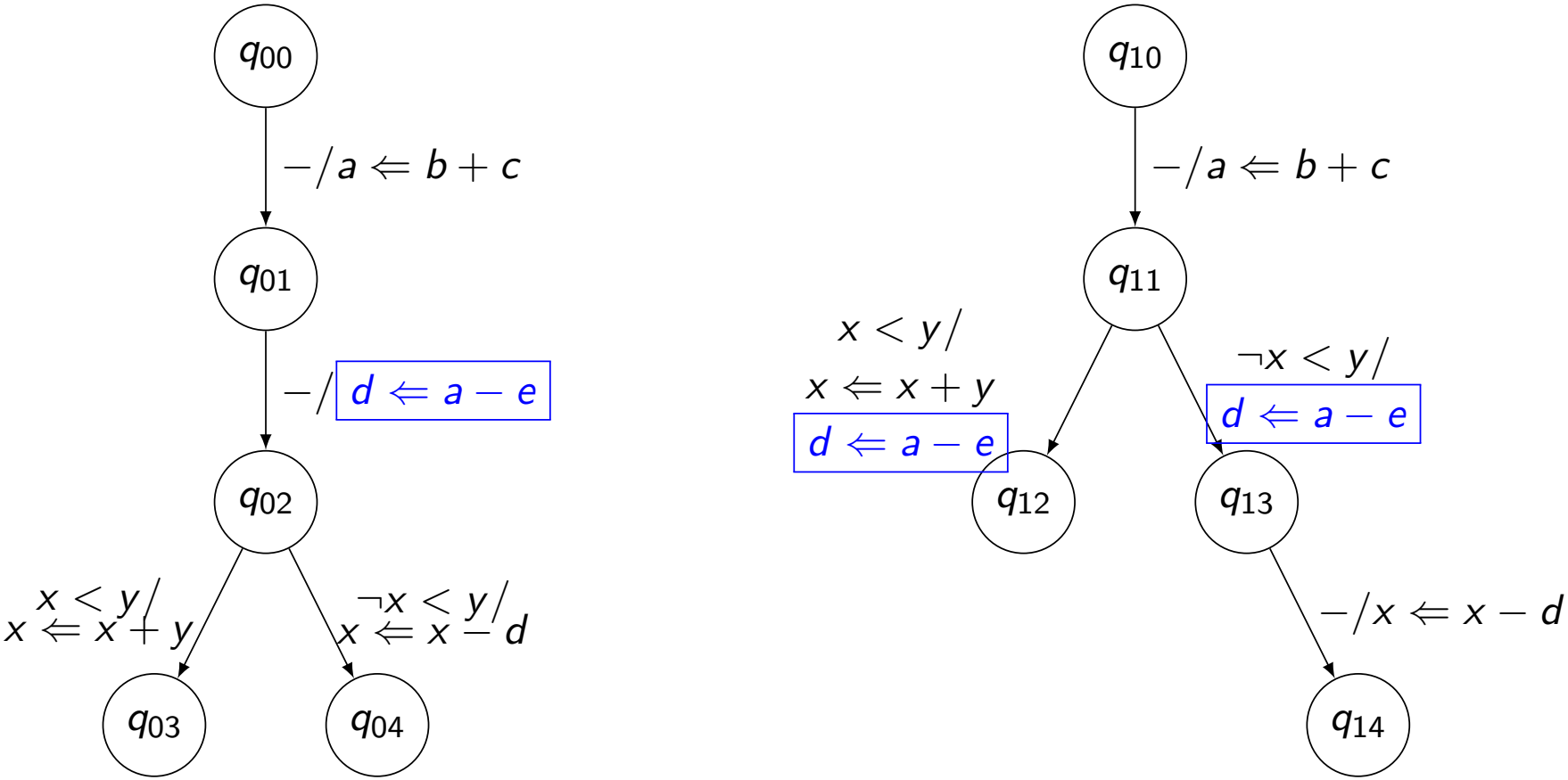
Related Work

K. Banerjee et al, "Verification of code motion techniques using value propagation", TCAD, (2014).[VP Method]

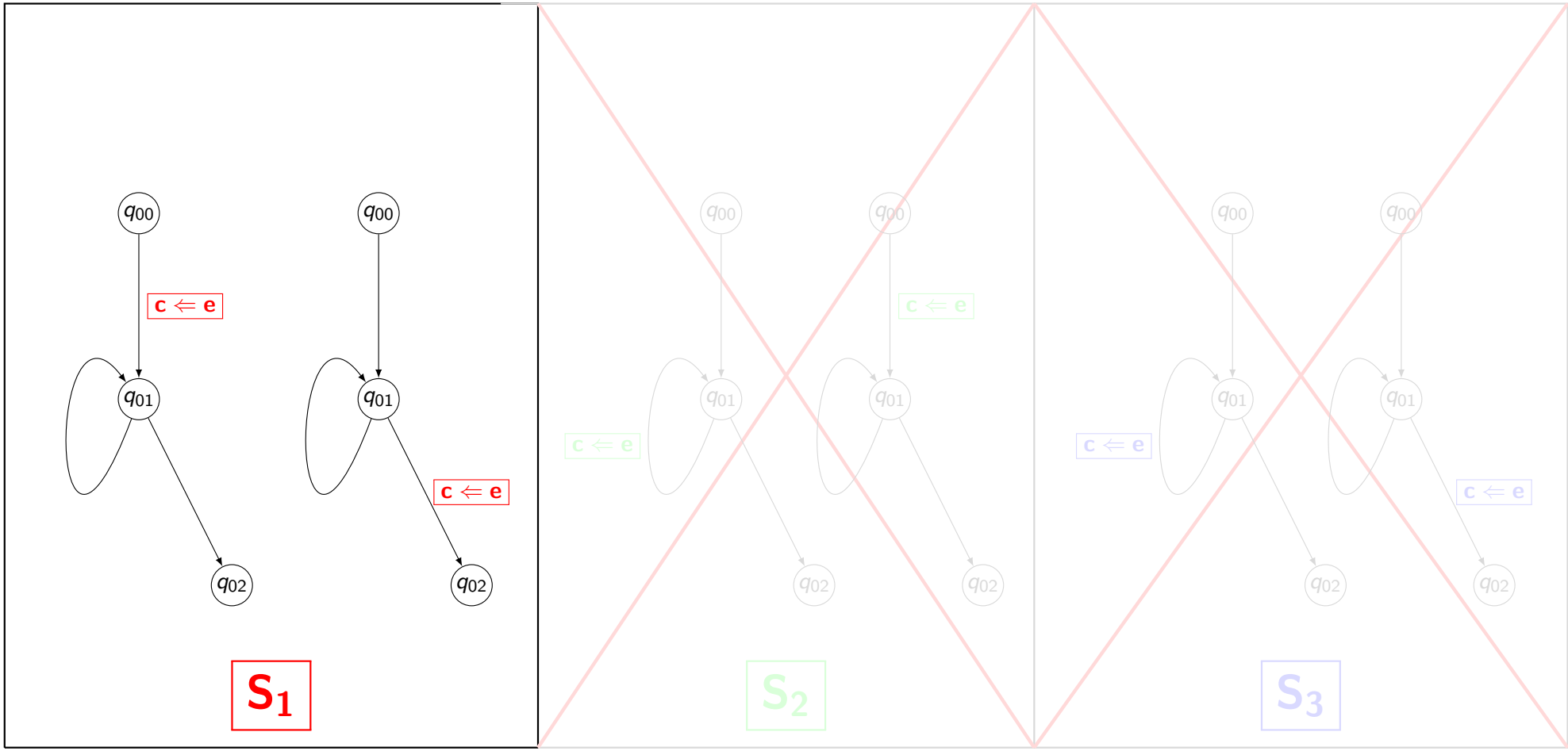
- Unified verification approach for code motion techniques.
- Uses value propagation method.
- Able to handle code motion across loop bodies.

Code Motion Based Transformations Techniques Ex.

Duplicating Down

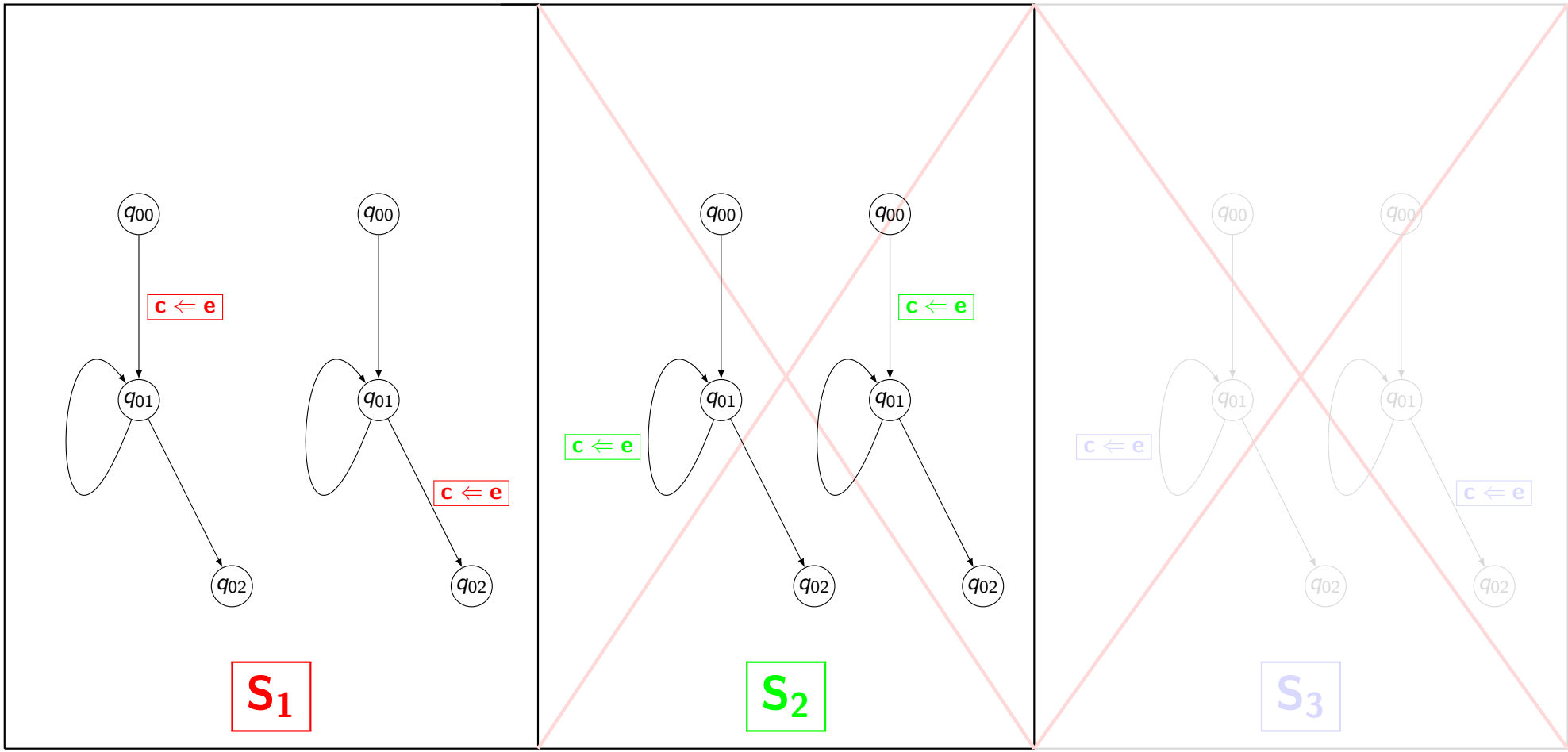


Code Motion Involving Loops



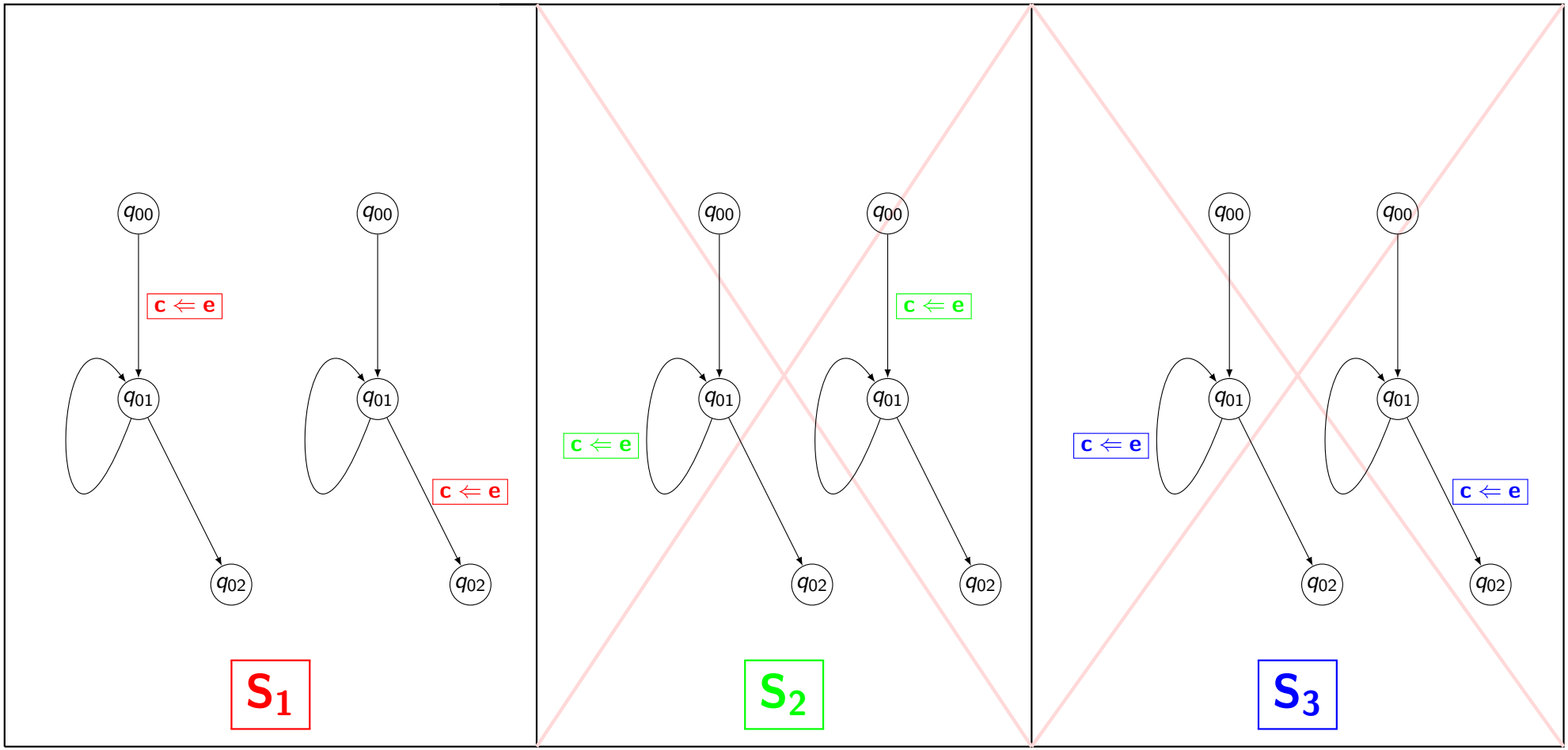
S_1 Some code segment before a loop body is placed after the loop body or vice versa (i.e. code motion across loops).

Code Motion Involving Loops



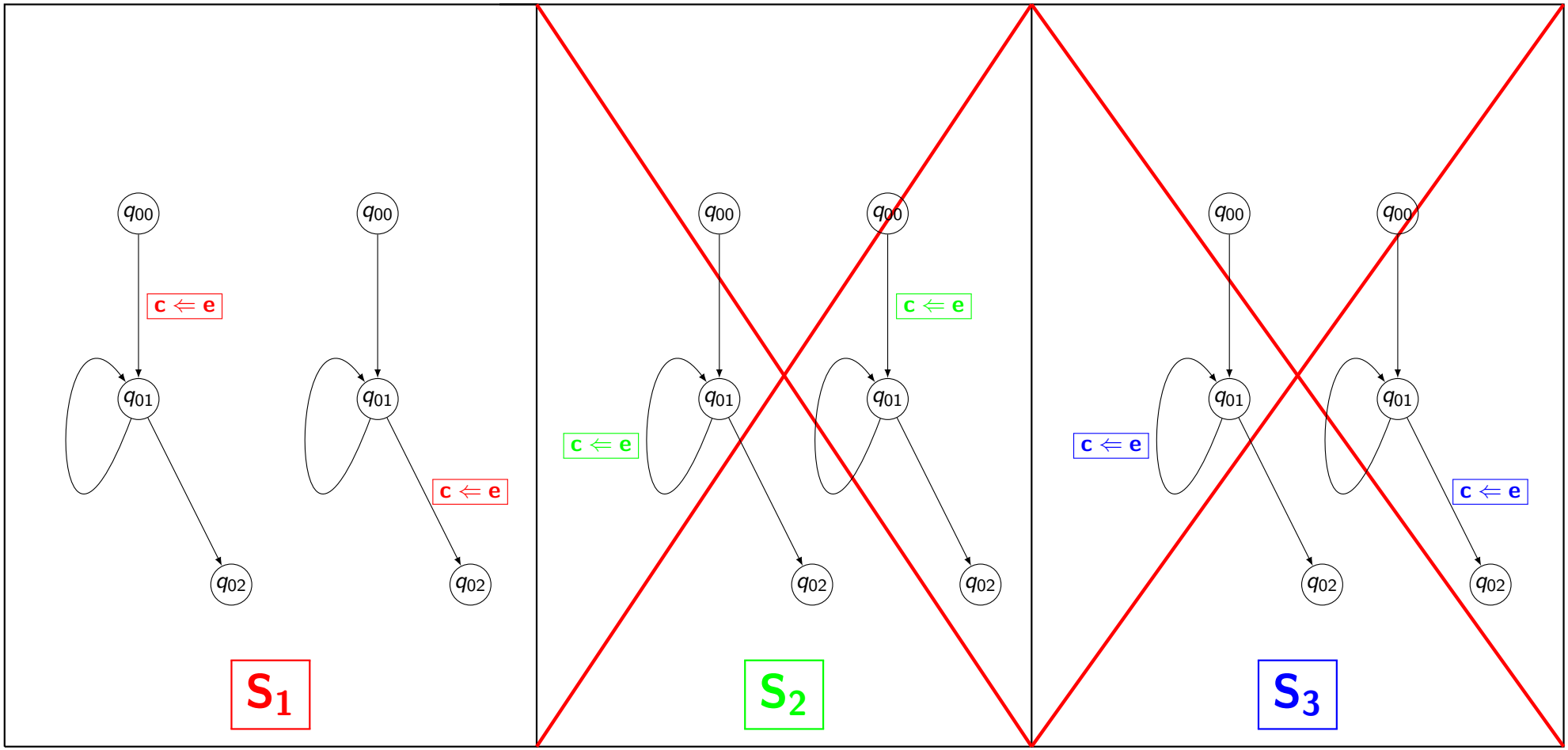
S_2 Some code segment is moved before the loop from inside the loop body.

Code Motion Involving Loops



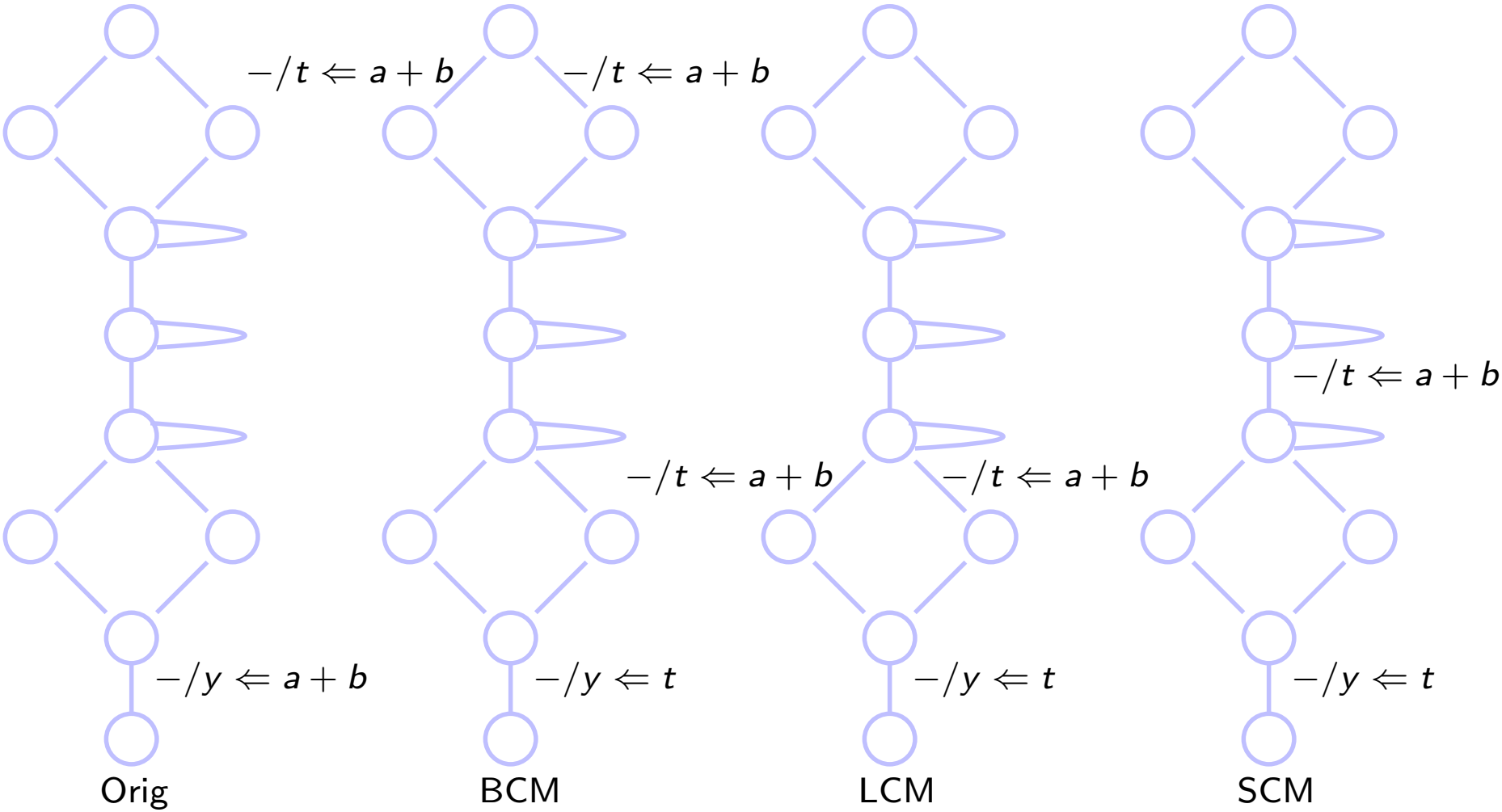
S_3 Some code segment is moved after the loop from inside the loop body.

Code Motion Involving Loops



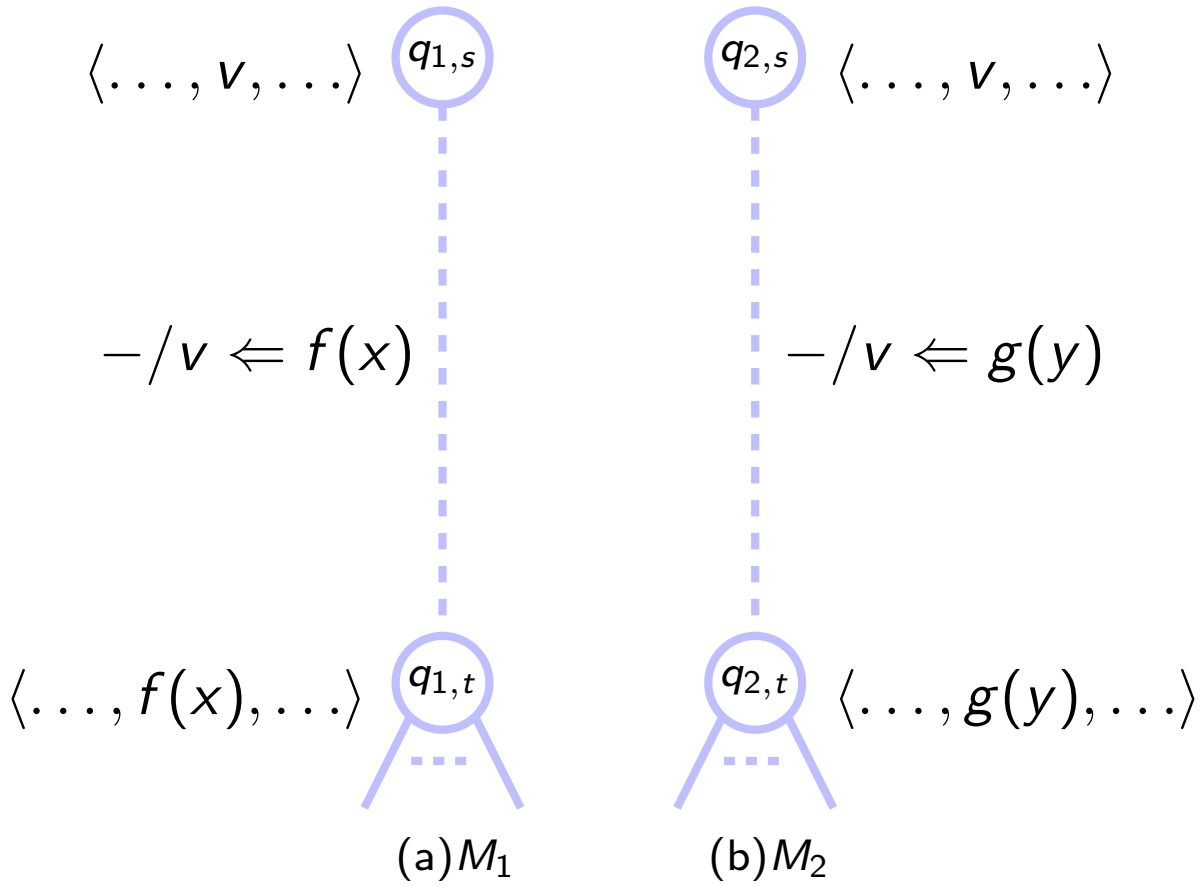
- R. Chouksey, C. Karfa and P. Bhaduri, Translation Validation of Code Motion Transformations Involving Loops, IEEE TCAD, vol. 38, no. 7,

A major challenge: Code motions across loops



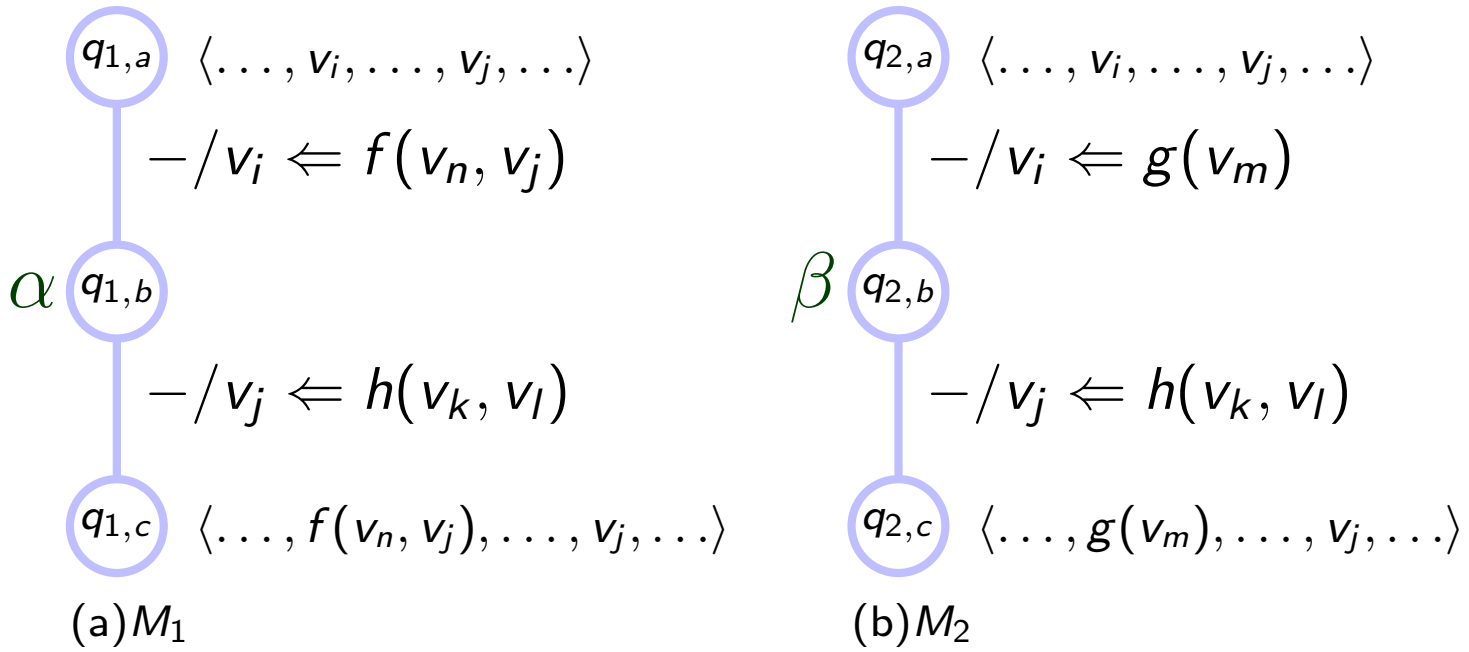
A path, by definition, cannot be extended beyond a loop.

The method of symbolic value propagation



An example of value propagation

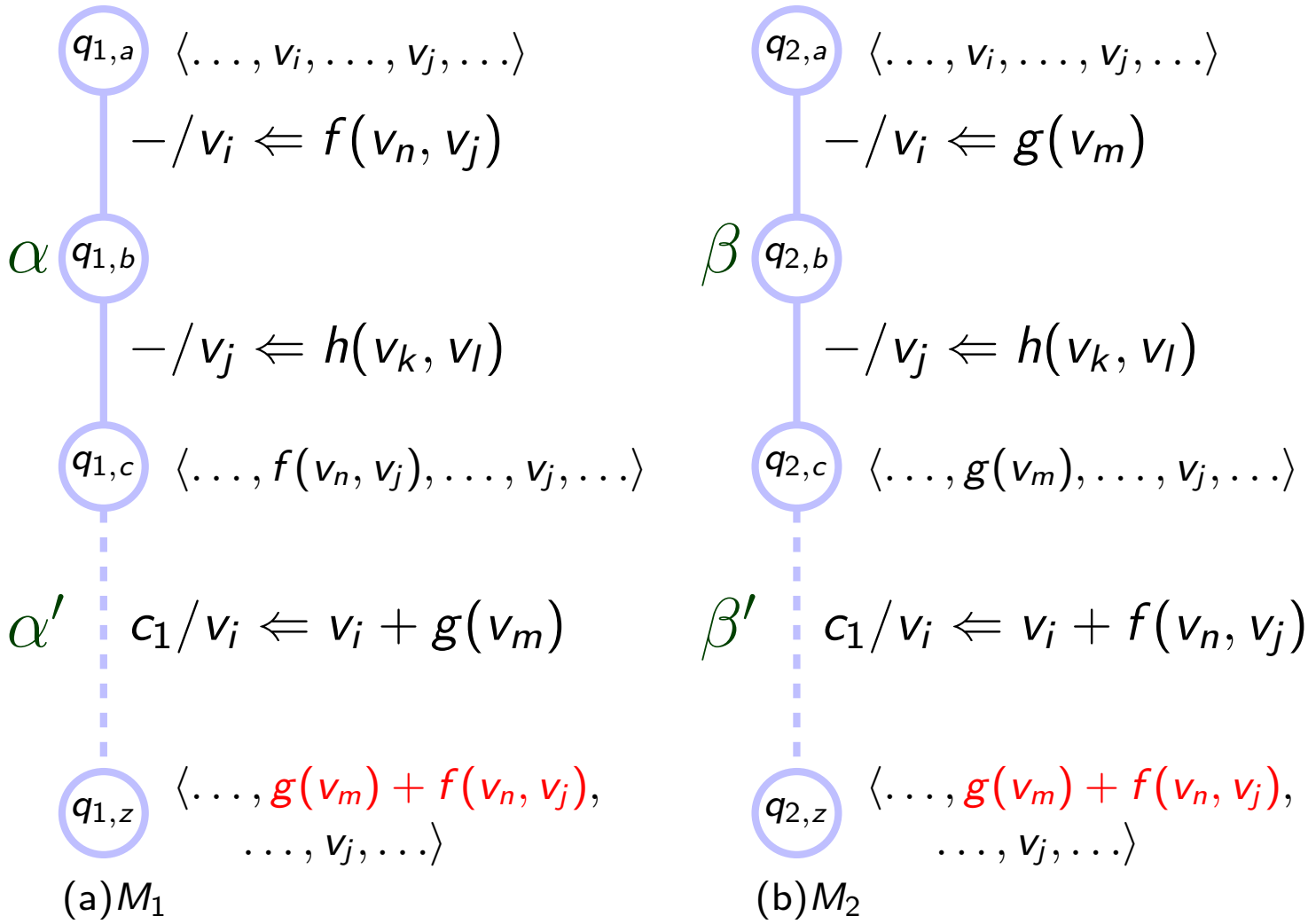
The method of value propagation



(a) M_1 (b) M_2

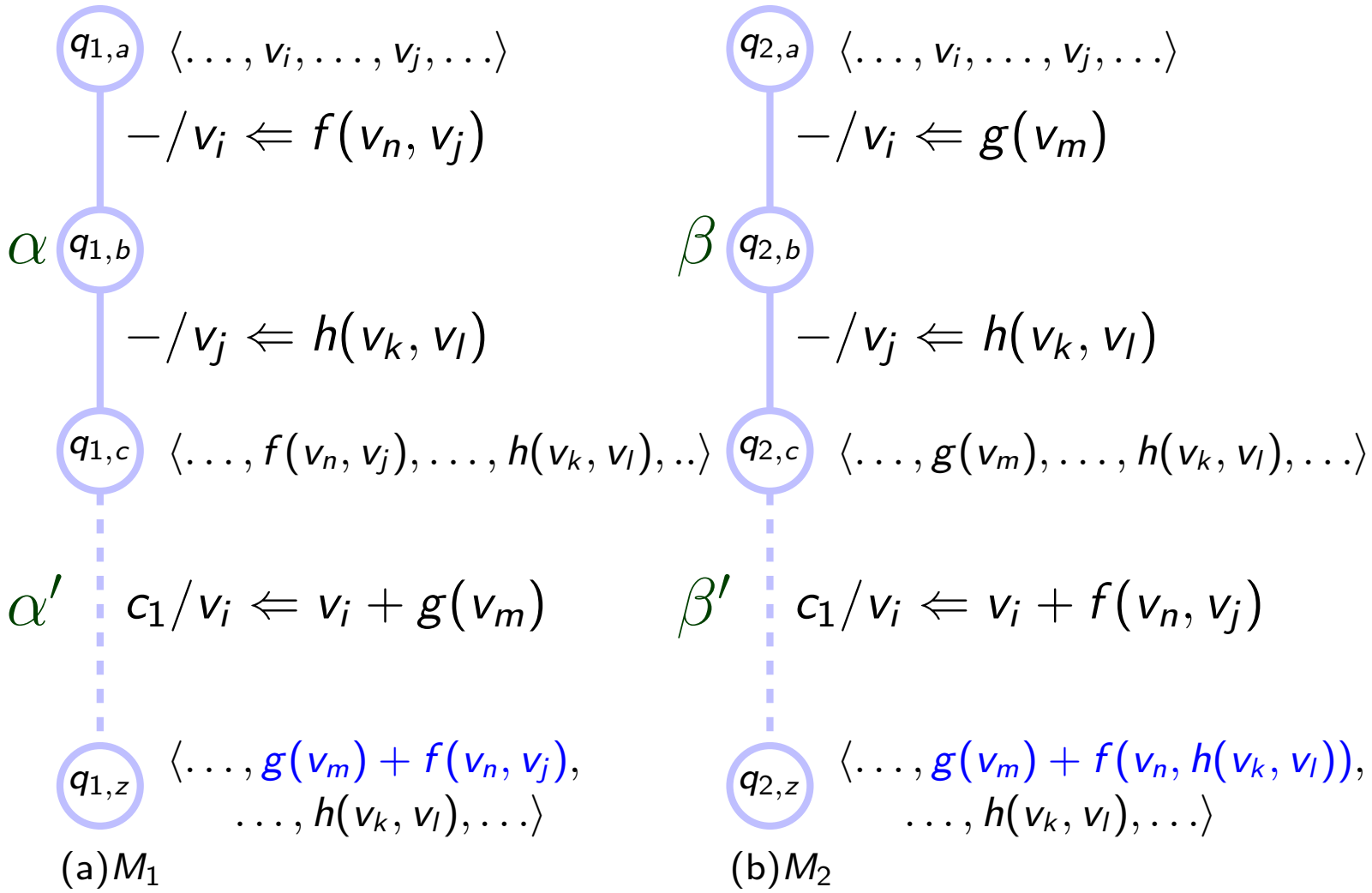
An example of value propagation with dependency between propagated values

The method of value propagation



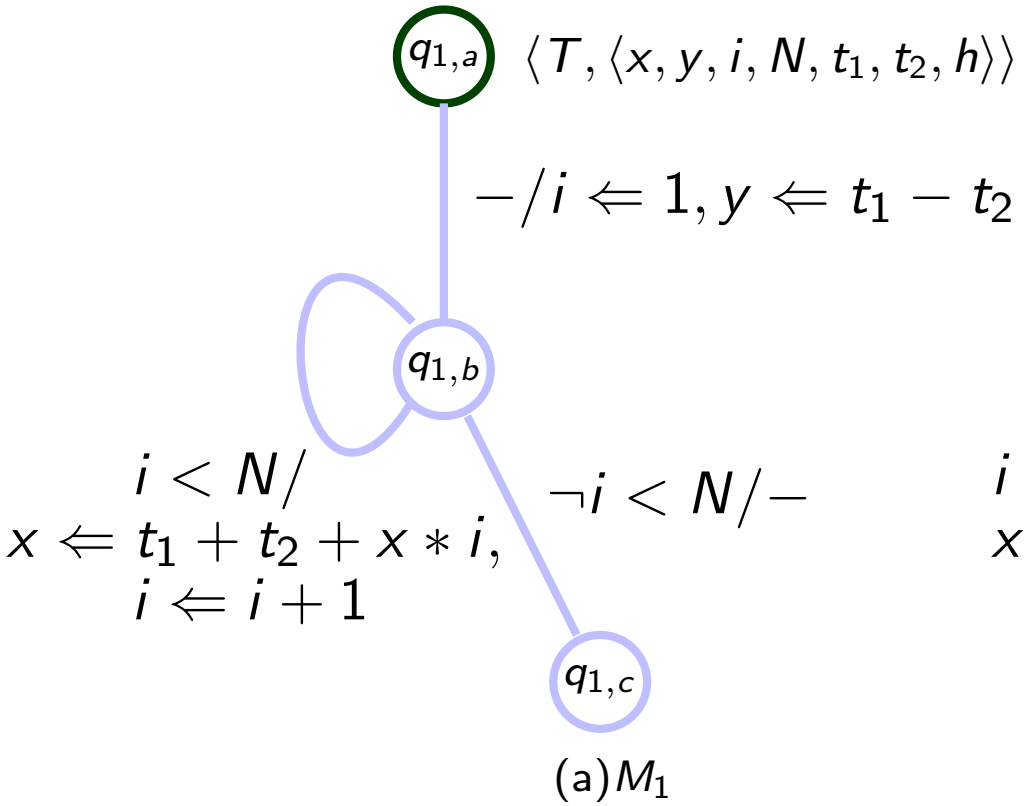
An *erroneous* decision taken

The method of value propagation

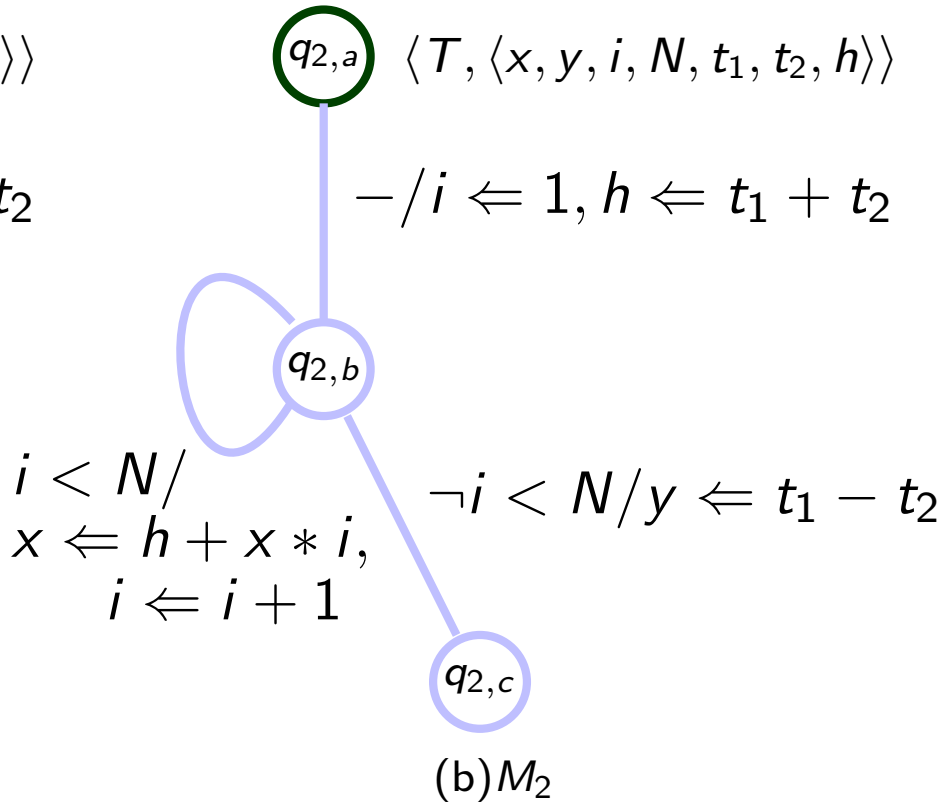


Correct decision taken

Equivalence checking using value propagation



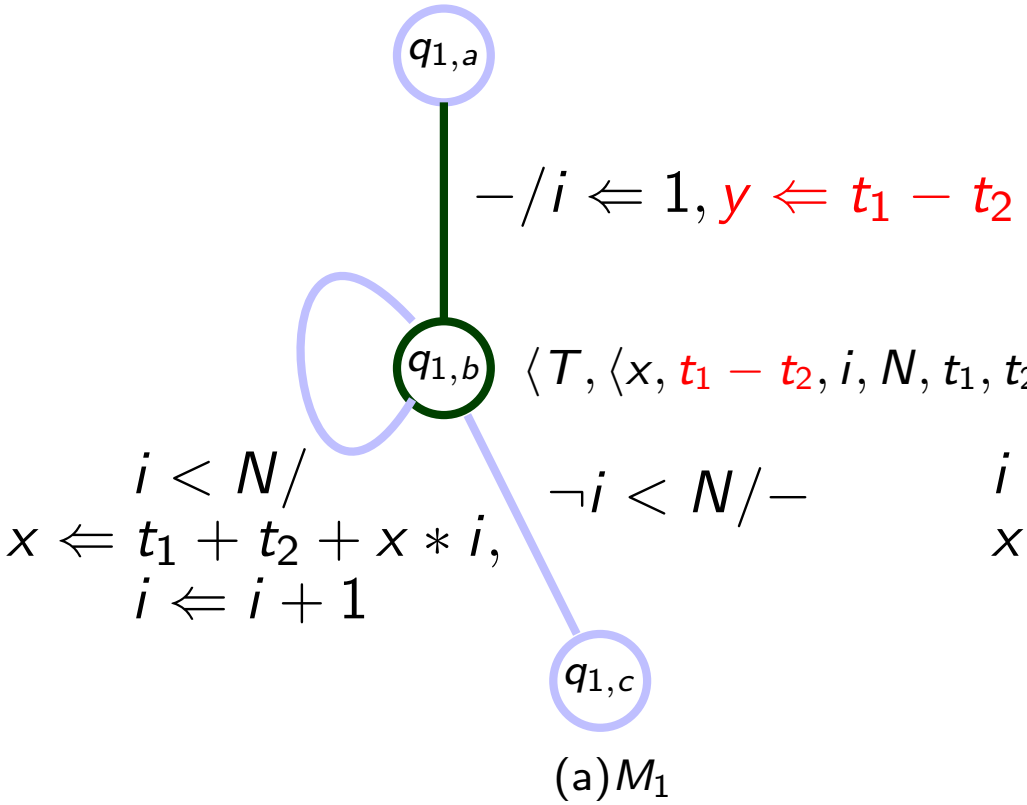
(a) M_1



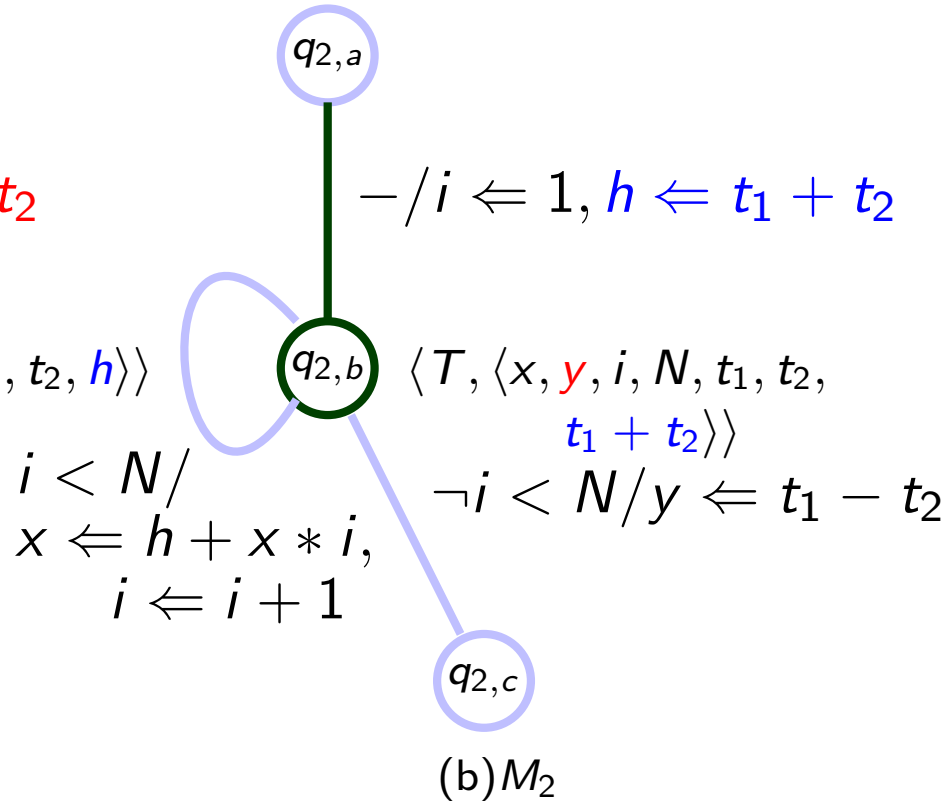
(b) M_2

At the reset states

Equivalence checking using value propagation



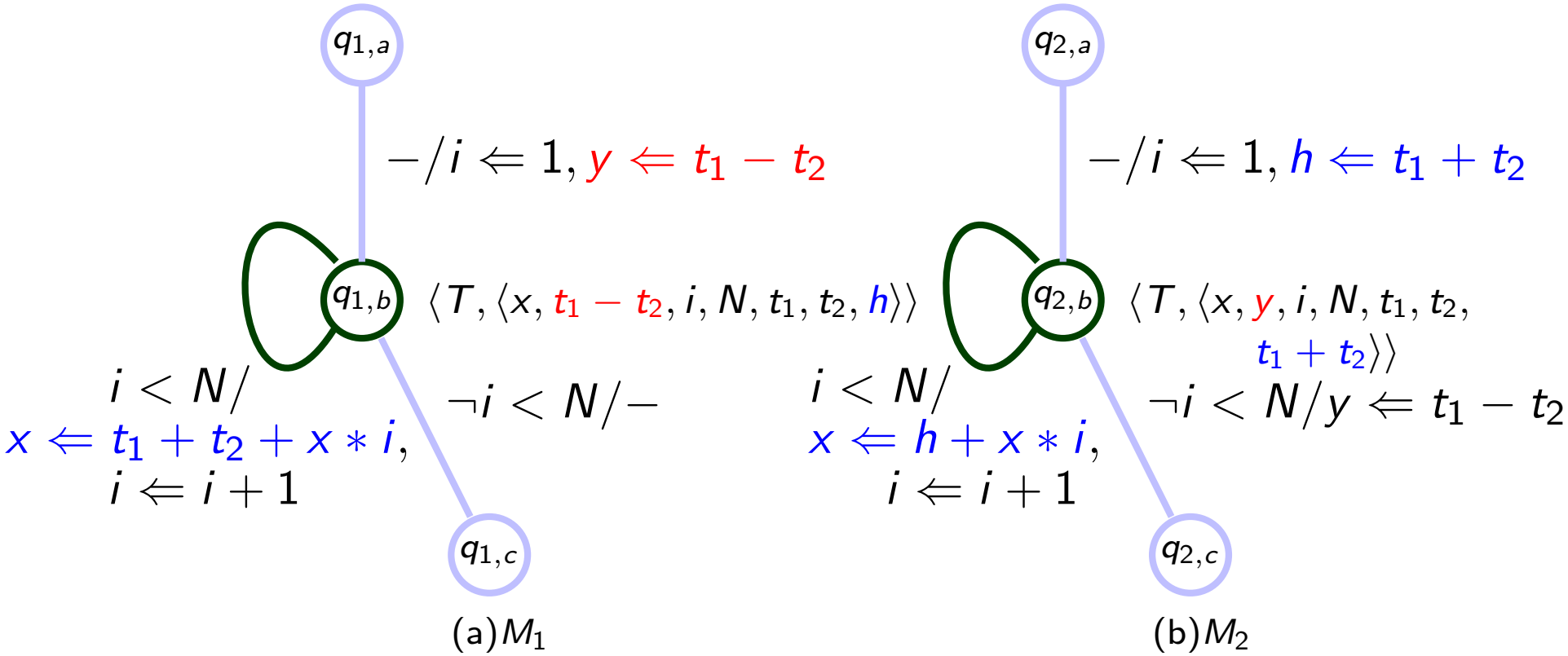
(a) M_1



(b) M_2

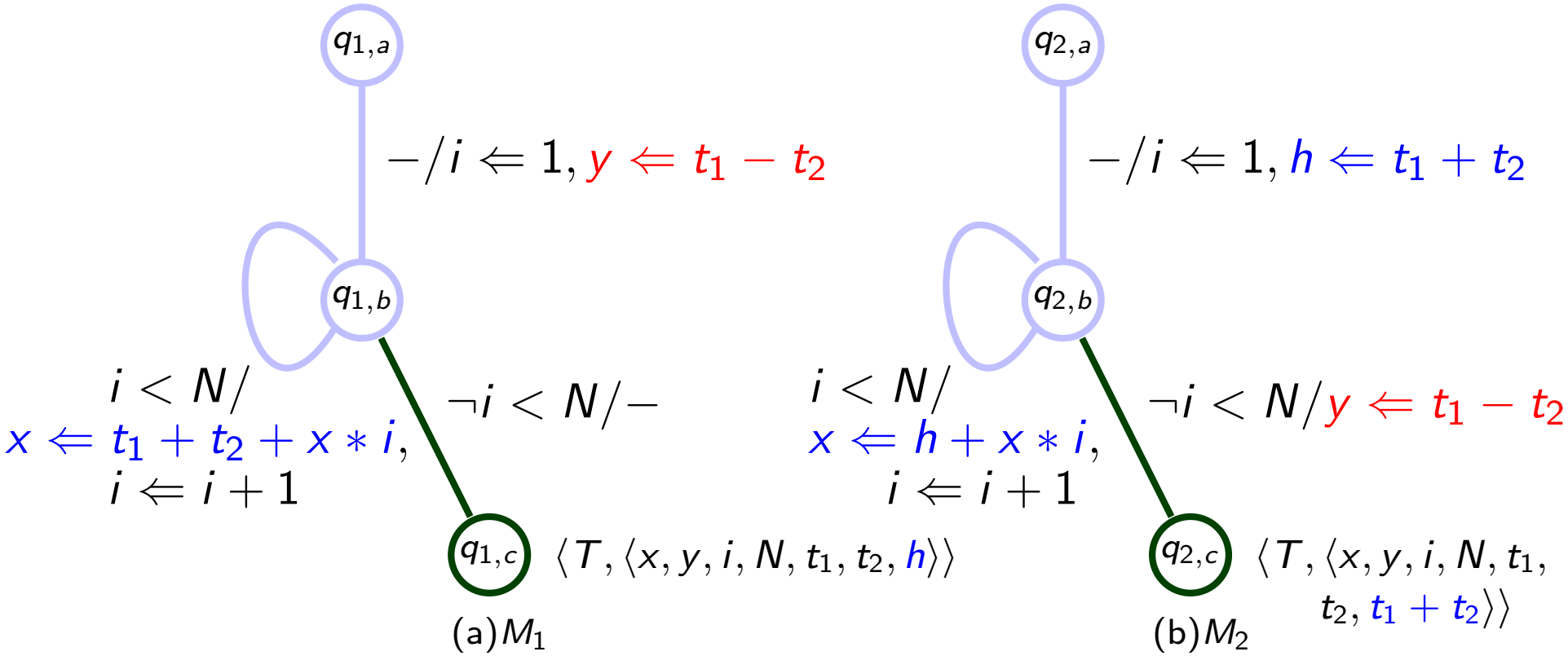
At the beginning of the loops

Equivalence checking using value propagation



At the end of the loops

Equivalence checking using value propagation



At the end states

Thank you!